

AJAX

Lorenzo Marconi

Tecnologie e Sistemi Web

Corso di laurea in Ingegneria informatica e automatica

Sapienza Università di Roma

a.a. 2025/2026



SAPIENZA
UNIVERSITÀ DI ROMA

AJAX

- **AJAX** = Asynchronous Javascript and XML
- Tecnologia basata su JavaScript e sull'**interazione asincrona** tra web client e web server
 - il web client fa una richiesta al server ma, a differenza delle altre forme di interazione previste, **NON** si interrompe fino all'arrivo della risposta (response) da parte del server
- Inoltre è possibile (ri)caricare anche solo una piccola parte della pagina attualmente visualizzata dal browser: ciò comporta maggiore velocità di esecuzione del browser e maggiore fluidità nell'interazione con l'utente

AJAX

- XML è il formato inizialmente previsto per lo scambio dei dati tra client e server
- Tuttavia, i dati possono essere scambiati anche in altri formati (JSON o altro)
- Gli oggetti **XMLHttpRequest (XHR)** modellano la comunicazione asincrona tra client e server
- Per attivare tale tipo di comunicazione, il client deve creare un nuovo oggetto XHR ed usare gli attributi e i metodi di questo oggetto

Oggetto XHR

readyState	0 (UNSENT) 1 (OPENED) 2 (HEADERS_RECEIVED) 3 (LOADING) 4 (DONE)
status	Codice di stato HTTP. Esempi: • 200 = ok • 404 = page not found
timeout	Per impostare un tempo massimo d'attesa (in ms)
onreadystatechange	Specifica una funzione invocata quando cambia l'attributo readyState (quindi possibilmente più volte)
onloadend	Come onreadystatechange ma ignora i readyState intermedi
ontimeout	Specifica una funzione invocata allo scadere del timeout previsto nell'omonima proprietà

Oggetto XHR (segue)

abort()	Interrompe tutte le attività create dell'oggetto e lo resetta
open(mode, url, boolean)	mode: tipo della HTTP request (GET, POST, HEAD...) url: indirizzo del file boolean: true (asincrona) / false (sincrona)
send(data)	Invia dei dati (data potrebbe essere una stringa o altro). Se la request è GET, data deve essere vuoto o null. Causa una DOMException (INVALID_STATE_ERR) se readyState è diverso da 1
responseText	Contiene i dati caricati (stringa di caratteri). Assume il suo valore finale quando readyState diventa uguale a 4
responseXML	Contiene il documento XML caricato (oggetto XML document). È significativo solo se readyState == 4, altrimenti è null

AJAX: esempio

```
<!DOCTYPE html>
<html>
  <head>
    <title>AJAX: semplice esempio</title>
  </head>
  <body>
    <div>
      <button>Risorsa_1</button>
      <button>Risorsa_2</button>
      <button>Risorsa_3</button>
      <button>Risorsa_4</button>
    </div>
    <hr/>
    <div id="zonaDinamica">
      Seleziona con i bottoni la risorsa da visualizzare qui
    </div>
    <hr/>
    Resto del documento<br/>
    ...
```

AJAX: esempio (segue)

```
<script>
  var bottoni = document.getElementsByTagName("button");
  for (var i = 0; i < bottoni.length; i++) {
    bottoni[i].onclick = caricaRisorsa;
  }

  function caricaRisorsa(e) {
    var xhr = new XMLHttpRequest();
    xhr.onreadystatechange = gestisciResponse;
    xhr.open("GET", e.target.innerHTML + ".txt", true);
    xhr.send();
  }

  /* si noti l'uso della proprietà target degli eventi,
  che restituisce l'elemento che ha causato l'evento */
```

AJAX: esempio (segue)

```
function gestisciResponse(e) {  
    if (e.target.readyState == 4 && e.target.status == 200) {  
        document.getElementById("zonaDinamica").innerHTML  
        = e.target.responseText;  
    }  
}  
</script>  
</body>  
</html>
```

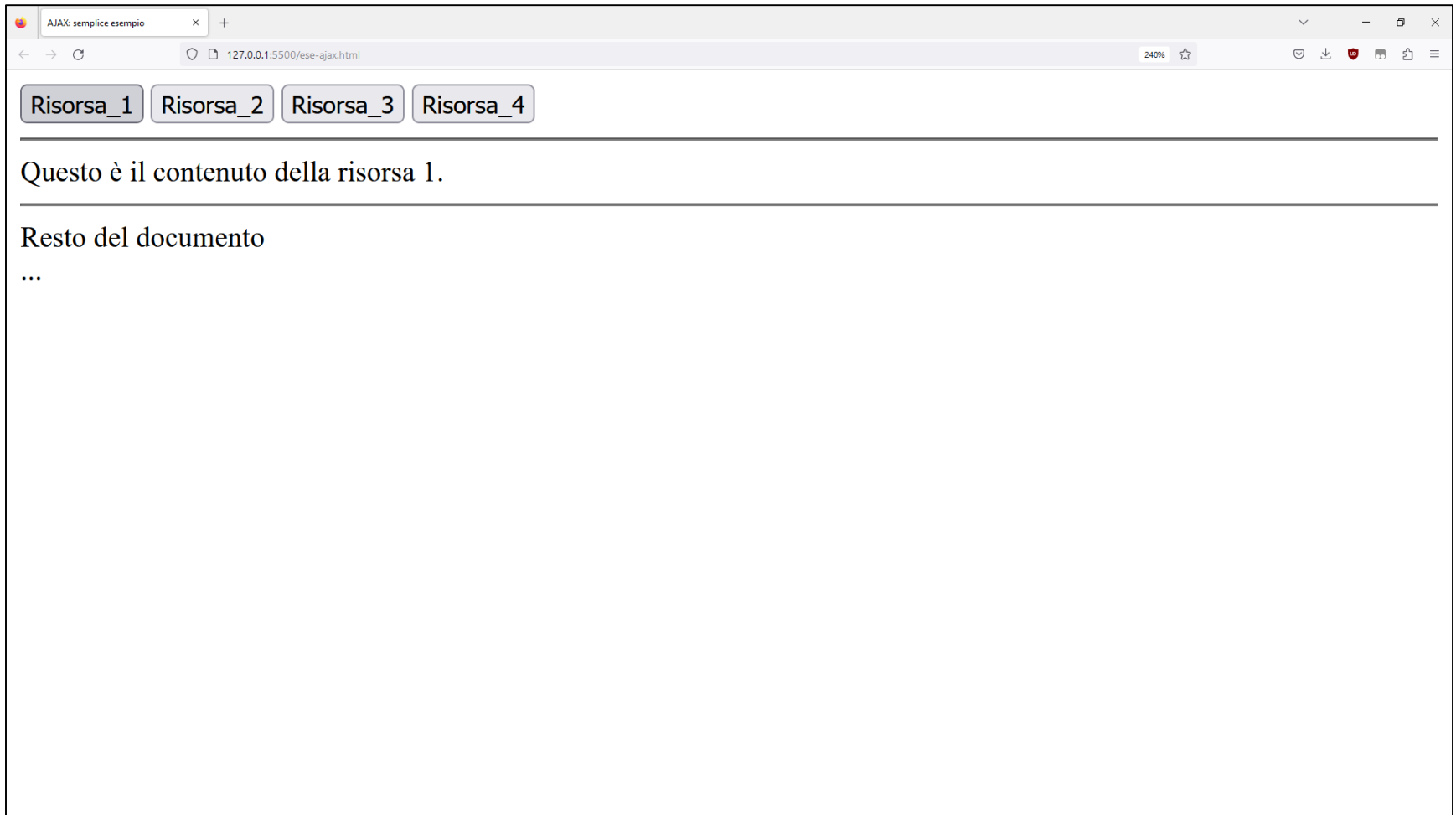

Esempio

Documento iniziale:



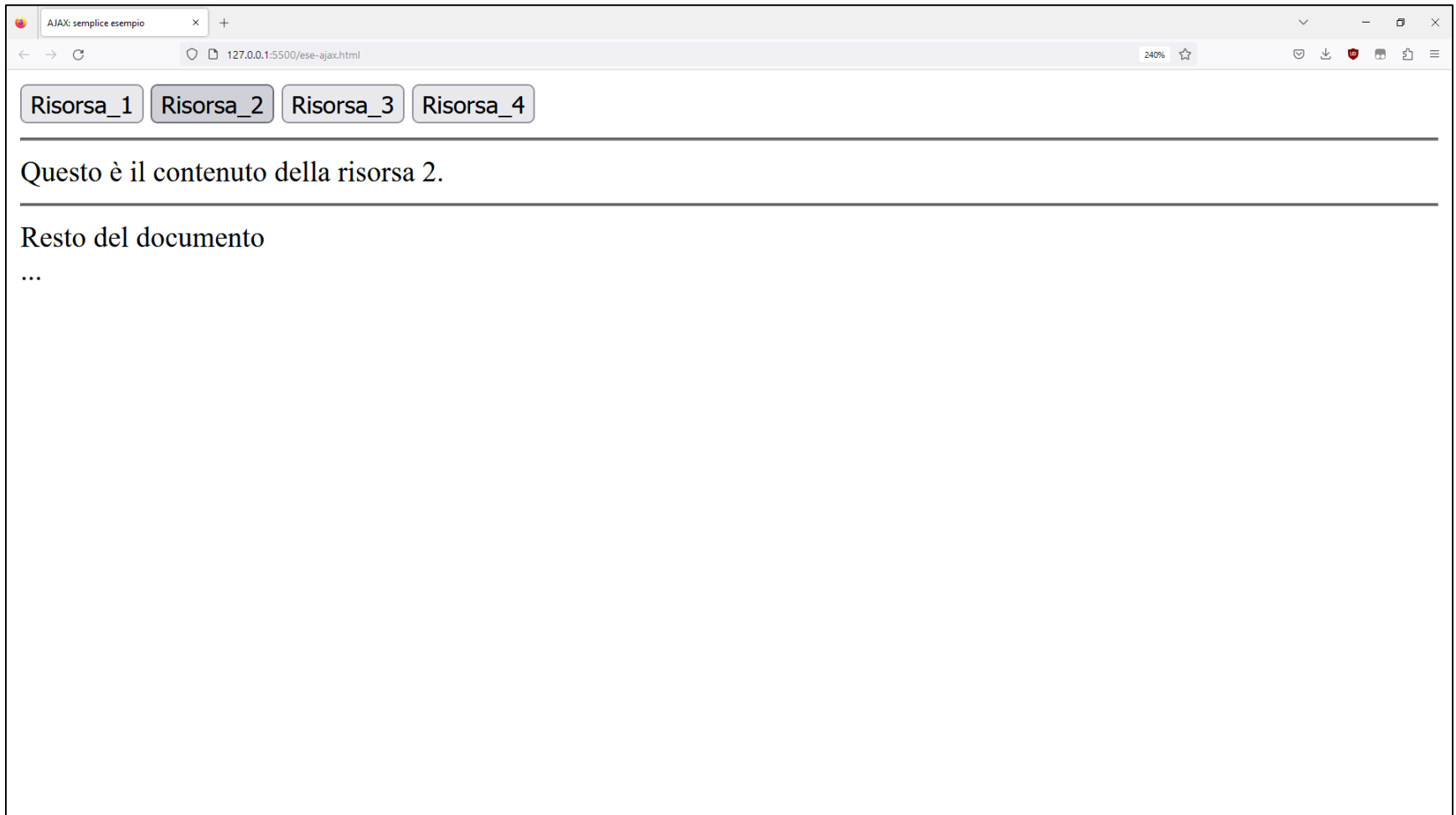
Esempio (segue)

Cliccando sul bottone Risorsa_1 si ottiene:



Esempio (segue)

Cliccando sul bottone Risorsa_2 si ottiene:



Esempio: commenti

- N.B.: ad ogni clic su un bottone, non viene ricaricato tutto il documento
- Solo il contenuto del tag `<div id="target">` del documento iniziale viene modificato dall'interazione asincrona con il server
- Ricordiamo che ad ogni invocazione di funzione associata ad un evento viene automaticamente passato un argomento corrispondente all'evento stesso (argomento e nelle dichiarazioni delle funzioni dell'esempio)
- L'evento ha l'importante proprietà **target** che contiene l'oggetto XHR stesso

Interazione sincrona

- Nel caso di open **sincrone** (attributo `boolean=false`), l'interprete JavaScript si ferma fino a che non viene ricevuta risposta dal server
- per questo le *open* sincrone vanno in genere evitate o comunque limitate

Caricamento di documenti XML

- AJAX e l'oggetto XMLHttpRequest trattano in modo speciale il caso in cui la risorsa oggetto della comunicazione tra client e server sia un documento XML
- In tal caso, il documento è memorizzato, sottoforma di oggetto XML document, nell'attributo **responseXML** (e non in responseText)
- per manipolare tale oggetto si possono usare i metodi del Document Object Model (DOM) di XML

Esempio con risorsa XML

Supponiamo ora che i precedenti documenti da caricare siano file XML.
Supponiamo ad esempio che il file Documento_1.xml sia:

```
<?xml version="1.0"?>
<radiceDocumento>
  <titolo>
    Documento 1
  </titolo>
  <contenuto>
    ...
  </contenuto>
</radiceDocumento>
```

Esempio con risorsa XML (segue)

(stessa struttura per i file Documento_2.xml, Documento_3.xml e Documento_4.xml)

In tal caso, questi documenti verrebbero ricevuti nell'attributo responseXML.

La funzione caricaRisorsa andrebbe modificata nel seguente modo:

```
function caricaRisorsa(e) {  
    var xhr = new XMLHttpRequest();  
    xhr.onreadystatechange = gestisciResponse;  
    xhr.open("GET", e.target.innerHTML + ".xml", true);  
    xhr.send();  
}
```


Esempio con risorsa XML (segue)

La funzione `gestisciResponse` andrebbe modificata come segue:

```
function gestisciResponse(e) {  
    if (e.target.readyState == 4 && e.target.status == 200){  
        var resp = e.target.responseXML;  
        var x = resp.getElementsByTagName("titolo");  
        document.getElementById("zonaDinamica").innerHTML=  
            "<h1>" + x[0].childNodes[0].nodeValue + "</h1>";  
    }  
}
```

(viene visualizzato il contenuto testuale dell'elemento titolo del file XML)

Same Origin Policy

- Anche AJAX è soggetto alla **Same Origin Policy** (politica della stessa origine): l'interazione può avvenire solo all'interno della stessa "origine"
- Due risorse hanno la stessa origine se le loro URI sono tali che lo **schema URI**, il **nome host**, e il **numero di porta** sono gli stessi
- Questa politica impedisce pertanto richieste "cross-domain" da parte del client nell'interazione AJAX
- Lo standard **CORS** (cross-origin resource sharing) consente di superare la Same Origin Policy, permettendo al browser di interagire preventivamente col server per stabilire se una richiesta cross-domain è sicura

Fetch API: la funzione `fetch()`

- Le **Fetch API** sono un'alternativa più moderna (e più semplice) all'uso di oggetti XHR
- Questa metodologia è basata sull'uso della funzione `Window.fetch()`, che ritorna una promise
- Esempi:
// ritorna una promise con l'oggetto la cui rappresentazione JSON si trova all'indirizzo specificato nella variabile 'url'
`fetch(url).then(response => response.json())`

// ritorna una promise con il testo restituito dalla risorsa 'url'
`fetch(url).then(response => response.text())`

Oggetto Response

Nella funzione passata al metodo `then()` si gestisce l'oggetto **response**, che possiede varie proprietà:

- `ok` : (true | false),
- `redirected` : (true | false),
- `status` : (es. 200),
- `url` : (URL da cui è stata effettivamente scaricata la risorsa)

oltre a vari metodi, tra cui:

- `json()`
- `text()`
- `blob()`

Fetch API: la funzione fetch() (segue)

Oltre all'URL, alla funzione fetch è possibile passare un oggetto con le **opzioni** relative alla richiesta HTTP, ad es.:

```
const jsonData = JSON.stringify({ utente: "esempio" });
fetch(url, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: jsonData,
  // ...
});
```

URLSearchParams

L'oggetto **URLSearchParams** è una soluzione molto semplice per inserire coppie chiave valore in un URL

```
const params = new URLSearchParams();  
params.append("nome", "Mario");  
  
// Chiamata GET a https://example.org/login?nome=Mario  
const response = await  
    fetch(`https://example.org/login?${params}`);
```

Un altro metodo molto utile per scrivere gli URL è **encodeURIComponent()**, che codifica i caratteri speciali della stringa passatagli in input